Tiago Autonomous Exploration and Manipulation

Autonomous and Mobile Robotics - Project

Francesco Cenni Giulio Mazzaroppi Giacomo Pezzolati

francesco.cenni3@studio.unibo.it
giulio.mazzaroppi@studio.unibo.it
giacomo.pezzolati@studio.unibo.it

University of Bologna

Academic Year 2025/2026

February 7, 2026

Overview

1. Introduction
2. Task 1: Exploration
3. Task 2: Navigation
4. Task 3: Pick and Place

Project Overview

- **Objective:** Develop a complete autonomous system for the TiaGo robot combining navigation and manipulation skills.
- **Tasks:**
  1. Map Generation (SLAM).
  2. Autonomous Localization & Exploration.
  3. Pick and Place of specific ArUco cubes.



Figure 1: Map Generation in RViz.



Figure 2: Autonomous Localization & Exploration



Figure 3: Pick and Place of specific ArUco cubes

System Architecture & Design

Development Stack

- **Framework:** ROS2 Humble Hawksbill.
- **Simulation:** Gazebo 11 (Physics & Environment).
- **Visualization:** RViz2 (Real-time sensor data & markers).

Architectural Pattern

We adopted a modular **Policy vs. Actuator** approach:

- **Main Policy (Coordinators):** High-level nodes managing the task logic and state machine.
- **Generalized Actuators:** Reusable skills (e.g., `map_navigator`, `head_motion`) agnostic of the specific task.
- **Handshake Protocol:** Coordination is handled via strictly defined Topics, Services, and Actions to ensure robust synchronization[a].

[a] Custom messages and service definitions are isolated in a dedicated **ament_cmake** package: `tiago_exam_interfaces`.

Package Organization: `tiago_exam`

The Project is entirely accomplished inside the `tiago_exam` package. This is a **ament_python** package structured as follows:

- **launch:** Contains three launch files to start each respective task environments.
- **tiago_exam:** Organized into sub-packages for each task and a shared utilities package.
- **data:** Contains the csv file for the aruco markers records used in task2 and task3.
- **config:** Stores RViz configuration files and map data for easy access across tasks.
- **temp:** Contains the debug map images for the exploration surface and reference frames.

Task 1: Autonomous Map Generation

Objective

Perform a full autonomous exploration of an unknown environment to generate a consistent 2D occupancy grid map using SLAM.

Environmental Challenges

The testing environment presents specific difficulties that standard configuration could not handle:

- **Complex Geometry:** Narrow passages and obstacles requiring precise maneuvering.
- **Lighting Artifacts:** Strong shadows created false positive obstacles in the LIDAR readings (ghost objects).



Figure 4: Environment in Gazebo 11.

Task 1: Exploration Strategy

To achieve efficient coverage, we integrated a C++ node based on `m-explore`, enhancing the standard frontier exploration logic.

Optimized Frontier Selection Logic

Instead of a greedy approach, we tuned the cost function parameters to prioritize frontiers that are:

1. **Closer** to the robot's current pose (minimizing travel time).
2. Located in **larger unexplored voids** (maximizing information gain).

Completion Workflow:
Upon exhaustion of the whitelist frontiers, the robot automatically navigates back to the **Home Position** and triggers the map saver, storing the result in the `maps/` directory.

Task 1: System Configuration & Safety

Safety Measure: Pre-Exploration Configuration

Before moving, the system executes a mandatory **Tuck Arm** procedure. Folding the arm into a safe configuration minimizes the robot's footprint, preventing collisions with low obstacles or door frames during autonomous navigation.

Sensor & Navigation Tuning

We refined the navigation stack parameters to address the environmental noise:

- **LIDAR Raytracing:** Adjusted strictly to filter out shadow reflections and prevent "ghost obstacles" on the map.
- **Motion Profile:** Smoothed acceleration limits to ensure stable mapping and prevent odometry drift.
- **Cost Map Parameters:**



Figure 5: Generated Occupancy Grid

Task 1: Exploration Video Result



Figure 6: Video Placeholder - Link to YouTube or local file.

Task 2: System Architecture & Orchestration

The complex behavior required for Task 2 is managed by a central **Coordinator Node** that acts as the high-level decision maker.

The Coordinator Design

- **Finite State Machine (FSM):** Strict transition logic between *Exploring*, *Tracking*, and *Inspecting* states.
- **Concurrency Handling:** Utilizes `ReentrantCallbackGroup` and `MultiThreadedExecutor`. This allows the node to process sensory interrupts (e.g., seeing a marker) immediately, even while blocking navigation actions are active.
- **Interrupt Logic:** The standard exploration loop is designed to be preempted instantly upon ArUco detection.



Figure 7: State Machine Logic

Task 2: Global Localization

Since the robot spawns in an unknown location, standard AMCL localization is insufficient immediately after startup. We implemented a robust initialization routine.

The Converge Strategy:

1. **Particle Spread:** Trigger `/reinitialize_global_localization` to distribute AMCL particles uniformly across the map.
2. **Data Gathering:** The robot executes a 360° in-place rotation at 1.0 rad/s to feed the laser scan matcher.
3. **Covariance Check:** The routine continuously monitors the pose covariance matrix, ensuring it falls below strict thresholds before allowing any movement.



Figure 8: AMCL Particle Convergence



Figure 9: Covariance Reduction Over Time

Task 2: Exploration Strategy

After the localization, the Coordinator Node activate the following nodes:

Generalized Nodes

- **Aruco Marker Detector:** The node continuously monitors the camera feed for the station markers.
- **Explorator:** The node processes the occupancy grid map to create a "whitelist" of safe navigation targets, ensuring the robot only attempts to navigate to locations that are free of obstacles.
- **Navigator:** The robot continuously samples from the "whitelist" of safe exploration goals. After reaching each goal, the robot performs a 360° rotation to improve its environmental awareness.



Figure 10: Clearance-Based Goal Sampling



Figure 11: Random Safe Goal Navigation

Task 2: Exploration Strategy - Contd.

Exploration Strategy

- **Tracking:** Once an ArUco marker is detected, the robot is brought to a specific distance from the marker, while continuously facing it by adjusting the camera tilt and pan angles.
- **Inspecting:** Once the robot is correctly positioned, it transitions to the Inspecting state, where the marker and robot Pose is saved in a csv file for later use.
- **Homing:** After all the markers are detected, the robot automatically navigates back to the Home Position.



Figure 12: Aruco Tracking Behavior



Figure 13: Aruco Inspecting Behavior



Figure 14: Homing Behavior

Task 2: Safety Navigation

Map Processing Pipeline

Using OpenCV on the occupancy grid map:

1. **Safety Mapping:** Applied `cv2.distanceTransform` to calculate the distance of every pixel to the nearest obstacle.
2. **Filtering:** Created a "Whitelist Mask" of pixels where:

$$d_{obstacle} > (R_{robot} + \text{Clearance}_{0.7m})$$

3. **Goal Selection:** Randomly samples coordinates from the whitelist and converts them from Image Frame (pixels) to World Frame (meters).

Result: The robot never attempts to navigate into walls or narrow traps.

Task 2: Navigation Video Result



Figure 15: Video Placeholder - Link to YouTube or local file.

Task 3: Coordination Strategy

The `coordinator_task3` node manages the full transport cycle.

- **Data Loading:** Loads exact Pick and Place locations saved in Task 2 from `marker_locations.csv`.
- **Sequence:** Enforces the mandatory order: Cube **63** followed by Cube **582**.
- **Visual Feedback:** Publishes `target_cube_frame` to TF for real-time visualization in RViz.

Task 3: MoveIt2 Integration

The `manipulator` node interfaces with MoveIt2 using the `RRTConnectkConfigDefault` planner.

- **Collision Avoidance:** Added virtual collision boxes (IDs 10-13) to prevent the arm from hitting the floor or robot base.
- **Safety:** Implemented a "Safe Joint Configuration" to retract the arm during navigation.

```
# From manipulator.py
self.safe_joints_position = [
    0.0, 0.20, -1.34, -0.20, 1.94, -1.57, 1.37, 0.00
]
```

Task 3: Robust Grasping (Gazebo)

To ensure stability in the simulation (as suggested in the guidelines), we implemented the **Link Attacher** plugin logic.

- **Approach:** The arm moves to a "Pre-Grasp" pose defined by a $-0.4m$ offset along the X-axis of the marker.
- **Attaching:** Once the gripper closes, we call the `/ATTACHLINK` service to rigidly bind the cube to the gripper link.

This prevents the object from slipping during the robot's navigation phases.

Task 3: Placement & Release

The placement pose is calculated dynamically relative to the detected grasp transform.

- **Logic:** We reuse the grasp orientation but apply a vertical offset to ensure the cube is released *on top* of the table.
- **Offset:** $+15cm$ $(0.15m)$ on the Z-axis.

```
# From coordinator_task3.py
release_frame = kdl.Frame(self.base_T_marker)
release_frame.p[2] += 0.15 # Raise Z by 15 cm
```

After placement, the `/DETACHLINK` service is called to release the object physics.